



Password Cracking with John the Ripper

Dictionary Attack Lab — Cracking Password-Protected PDFs

Prepared by	Sheikh Rasel Mehedi
Internship	NetworkWalks Internship
Mentor	Waqas Karim (CCIE)
Batch	B082
Week	03
Topic	Password Cracking with John the Ripper — Dictionary Attack on Password-Protected PDFs
Date	September 1, 2026

- **Linkedin:** <https://www.linkedin.com/in/obfsec-rasel/>
- **GitHub:** <https://github.com/obfsec-rasel>

Table of Contents

1. Objective.....	2
2. Lab Environment & Tools	3
3. Background: PDF Encryption and the \$pdf\$ Hash Format	4
4. Methodology	5
4.1 Extracting the Password Hash.....	5
4.2 Understanding the Attack — Browser Dictionary-Attack Simulator.....	5
4.3 Cracking with John the Ripper (Johnny GUI)	5
5. Results.....	6
6. Screenshots	7
7. Key Observations & Learning Outcomes.....	9
8. Mitigation & Defensive Recommendations	10
9. Conclusion	11



1. Objective

This report documents a password-cracking lab completed as part of the NetworkWalks cybersecurity internship. The objective was to recover the passwords protecting a set of encrypted PDF files using a dictionary attack — first understood conceptually through NetworkWalks' browser-based cracking simulator, then carried out for real using John the Ripper via the Johnny GUI front-end.



2. Lab Environment & Tools

- Host: personal Windows PC
- Cracking engine: John the Ripper 1.9.0-jumbo-1 (Windows build, cygwin 64-bit, AVX2), operated through the Johnny GUI
- Hash extraction: Online HashCrack — a web-based converter that pulls the John-compatible \$pdf\$ hash out of a password-protected PDF
- Concept walkthrough: NetworkWalks' "Password Cracker through Dictionary Attacks" browser lab (networkwalks.com/password-cracker/)
- Wordlists: the simulator's built-in 100-password list and fasttrack.txt (221 words); a larger dictionary used for the real John the Ripper run
- Targets: multiple lab-issued password-protected PDF files, e.g. "My Locked PDF2.pdf"



3. Background: PDF Encryption and the \$pdf\$ Hash Format

PDF supports password-based encryption (RC4 or AES depending on the file's security-handler revision). Tools built around John the Ripper's pdf2john logic — including the Online HashCrack converter used here — read a locked PDF's encryption dictionary and output a single \$pdf\$-prefixed hash string encoding the revision, key length, permission flags, and the document's U/O security values. That string is everything John needs: it never has to touch the original PDF again, so the attack runs entirely offline. From there it is a plain dictionary attack — hash each candidate word the same way the PDF itself was hashed, and compare it to the stored value. A match recovers the password; anything else is unaffected.



4. Methodology

4.1 Extracting the Password Hash

Each locked PDF was uploaded to Online HashCrack's converter, which returned a \$pdf\$-format hash string ready for John the Ripper. The same result can be produced locally, without a third-party site, using the pdf2john script bundled with the Jumbo build of John:

```
| perl pdf2john.pl "My Locked PDF2.pdf" > pdf_hash.txt
```

4.2 Understanding the Attack — Browser Dictionary-Attack Simulator

Before running the real tool, a sample \$pdf\$ hash was pasted into NetworkWalks' web-based Password Cracker lab, which mirrors John's own logic: hash every word in a list and compare it to the target hash. The first run, against the built-in list / fasttrack.txt (221 words), returned ACCESS DENIED — the real password simply wasn't in that list. This reinforced the core limitation of any dictionary attack: it only succeeds if the actual password is present in the wordlist used.

4.3 Cracking with John the Ripper (Johnny GUI)

Johnny was pointed at the local john.exe (Jumbo build) executable, confirmed as John the Ripper 1.9.0-jumbo-1. Each extracted PDF hash file was then loaded and attacked in dictionary mode. The command-line equivalent of what Johnny runs under the hood:

```
| john --format=PDF --wordlist=<wordlist.txt> pdf_hash.txt
| john --show --format=PDF pdf_hash.txt
```

All three PDF challenges in the exercise were cracked this way, each recovered password shown directly in Johnny's Passwords tab and each success unlocking a NetworkWalks lab flag (see Results).



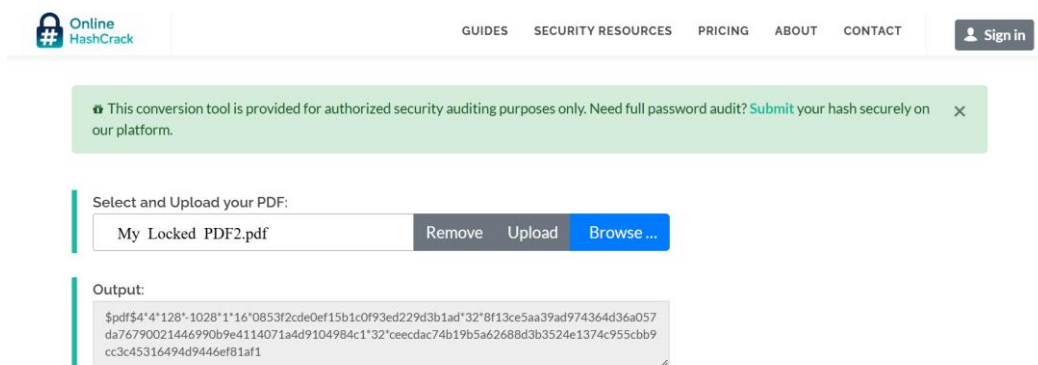
5. Results

Passwords and flags recovered across the three PDF challenges in this exercise:

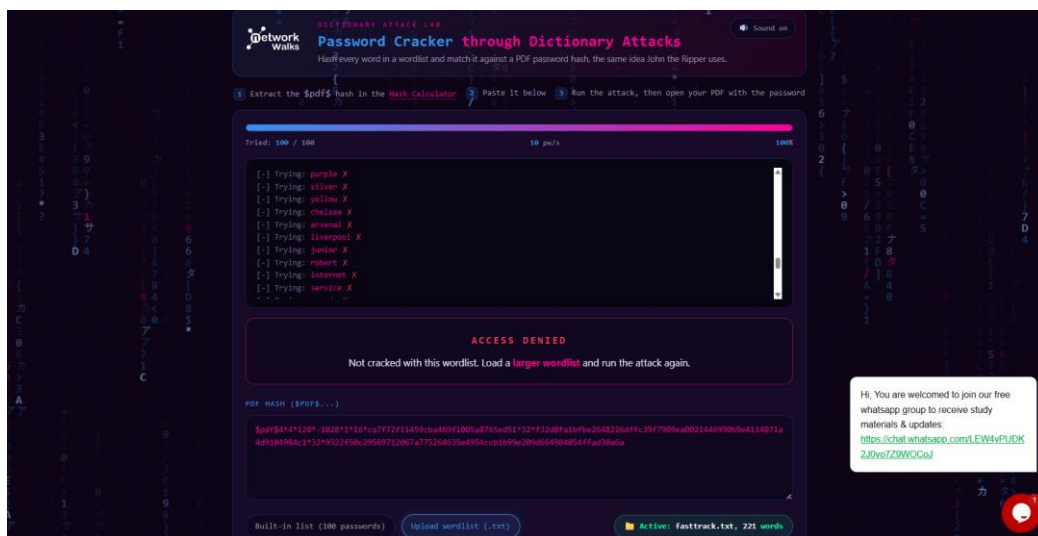
Target PDF	Method	Cracked Password	Flag Captured
PDF Challenge 1	John the Ripper (Johnny GUI), dictionary mode	good-luck	nw{networkwalks_flag_260821_1}
My Locked PDF2.pdf	John the Ripper (Johnny GUI), dictionary mode	password1	nw{cybersecurity_flag_captured_2608}
PDF Challenge 3	John the Ripper (Johnny GUI), dictionary mode	1qaz2wsx	nw{networkwalks_persistence_jtr_270521}



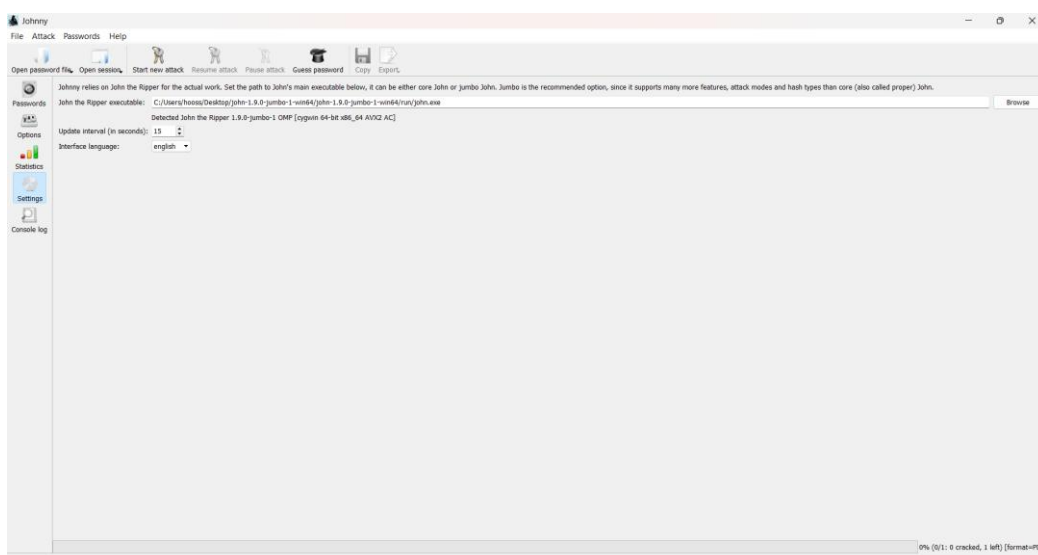
6. Screenshots



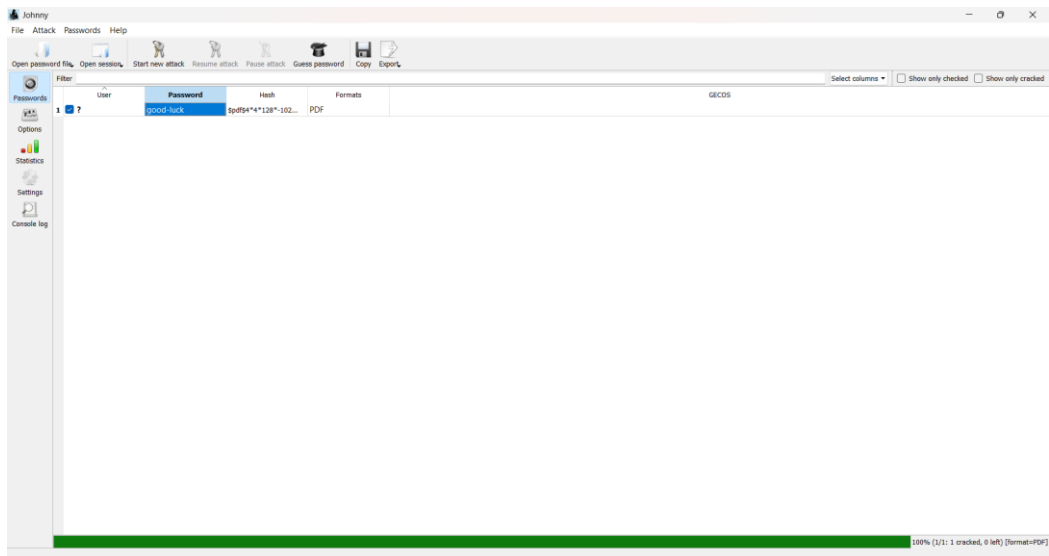
Online HashCrack — extracting the \$pdf\$ hash from "My Locked PDF2.pdf"



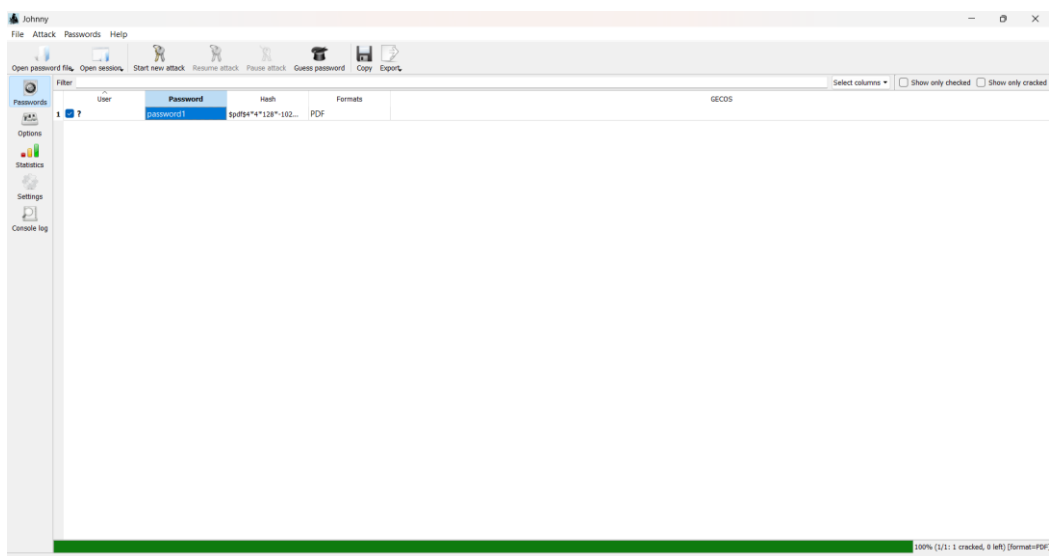
NetworkWalks browser dictionary-attack simulator — first attempt denied with a small wordlist



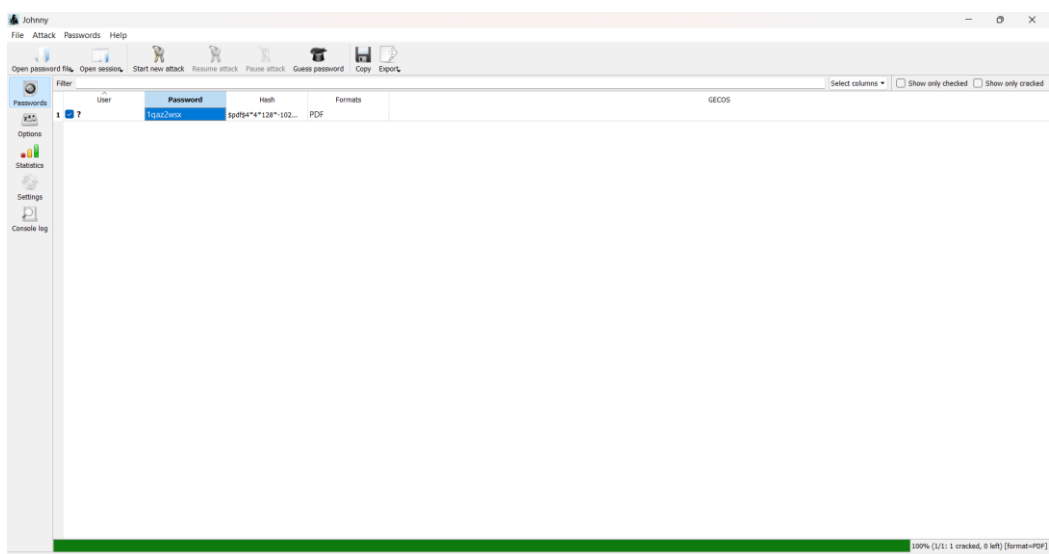
Johnny GUI configured with the local John the Ripper 1.9.0-jumbo-1 executable



PDF Challenge 1 cracked — password recovered: good-luck



My Locked PDF2.pdf cracked — password recovered: password1



PDF Challenge 3 cracked — password recovered: 1qaz2wsx

7. Key Observations & Learning Outcomes

- All three recovered passwords — good-luck, password1, 1qaz2wsx — were common, predictable strings, showing how quickly a basic dictionary attack defeats them once the right wordlist is used.
- The browser simulator's failed first attempt versus the real tool's success on the same style of hash illustrates that wordlist coverage, not the cracking tool itself, is usually the deciding factor.
- "1qaz2wsx" is a keyboard-walk pattern that looks complex but appears in virtually every large breach-derived wordlist — a reminder that visual complexity does not equal real entropy.
- Extracting the \$pdf\$ hash only reads the file's encryption dictionary; it doesn't modify or damage the PDF, which is why offline recovery attempts are fast and safe to test repeatedly.



8. Mitigation & Defensive Recommendations

- Avoid dictionary words and keyboard-walk patterns for PDF/document passwords; use a long, random passphrase instead.
- Where possible, use AES-256 (PDF 2.0 / ISO 32000-2) encryption rather than legacy RC4 or 128-bit schemes, which fall to offline attack much faster.
- Don't rely on a document password as the only control for sensitive files — pair it with access restrictions at the storage or sharing layer.
- Be cautious uploading real, sensitive PDFs to third-party "hash extractor" web tools outside a lab context — the service sees the file's contents, so this is appropriate for training data only, never for confidential documents.
- Use a password manager to generate and store strong, unique document passwords instead of memorable patterns.



9. Conclusion

This lab connected the theory of dictionary attacks — practiced first in NetworkWalks' browser simulator — to a real-world tool, John the Ripper, used through the Johnny GUI. Three password-protected PDFs were successfully cracked end to end: hash extraction, dictionary-mode attack, and recovery of the plaintext password, each confirmed by a captured lab flag. The exercise reinforced both the mechanics of offline PDF password recovery and the practical lesson that common, predictable passwords remain the weakest link regardless of the underlying encryption.

